UNIVERSIDAD NACIONAL DEL ALTIPLANO

PUNO

FACULTAD DE INGENIERÍA ESTADÍSTICA E INFORMÁTICA

ESCUELA PROFESIONAL DE INGENIERÍA ESTADÍSTICA E INFORMÁTICA

LENGUAJE DE PROGRAMACIÓN 1

Fundamentos de C y C++ aplicados a la Estadística

Material académico elaborado por:

DINA MILAGROS DELGADO QUISPE

Código: 247092

Tercer Semestre

2026

Índice general

Introducción **5**

1 Orígenes de la Computación y Programación **7**
1.1 Los Pioneros de la Computación 7
1.1.1 Charles Babbage (1791-1871): El Visionario Mecánico 7
1.1.2 Ada Lovelace (1815-1852): La Primera Programadora 7
1.1.3 Alan Turing (1912-1954): El Padre Teórico 7
1.1.4 John von Neumann (1903-1957): La Arquitectura Moderna . . . 8
1.2 Evolución de los Lenguajes de Programación 8
1.2.1 De las Máquinas a los Humanos 8
1.3 Lenguajes de Programación más Usados 9
1.3.1 Índices Estadísticos Internacionales 9
1.3.2 Lenguaje de Programación Líder 9
1.4 Paradigmas de Programación 12
1.4.1 Paradigma Imperativo 12
1.4.2 Paradigma Declarativo 12
1.4.3 Paradigma Orientado a Objetos (POO) 12
1.5 Lógica Proposicional 13
1.5.1 Operadores Lógicos Fundamentales 13

2 Introducción a C y C++ **15**
2.1 Relevancia de C y C++ en 2026 15
2.2 C: El Lenguaje del Sistema 15
2.2.1 Historia de C 15
2.2.2 Características de C 15
2.3 C++: Extensión de C 16
2.3.1 Historia de C++ 16
2.3.2 Diferencias Clave entre C y C++ 16
2.4 Primer Programa en C 16
2.5 Primer Programa en C++ 17
2.6 Tipos de Datos Fundamentales 17
2.6.1 Ejemplo con Diferentes Tipos 18

3 Operadores **19**
3.1 Operadores Aritméticos 19
3.1.1 División Entera 19
3.1.2 El Operador Módulo 20
3.2 Operadores Relacionales y Lógicos 20
3.2.1 Operadores Relacionales 20

3.2.2 Operadores Lógicos 21
3.2.3 Aplicación Estadística 21
3.3 Operadores de Incremento y Decremento 22
3.3.1 Prefijo vs Postfijo 22
3.3.2 Uso en Bucles 23
3.4 Operadores de Asignación Compuesta 23
3.4.1 Cálculo de Estadísticas 23

4 Estructuras de Control 25
4.1 Introducción 25
4.2 Estructuras Selectivas 25
4.2.1 if: La Decisión Simple 25
4.2.2 if-else: Dos Caminos 26
4.2.3 if-else if-else: Decisiones Múltiples 26
4.2.4 switch: Opciones Discretas 27
4.3 Estructuras Repetitivas (Bucles) 28
4.3.1 El Bucle while 28
4.3.2 El Bucle for 29
4.3.3 El Bucle do-while 29
4.3.4 Comparación de Bucles 30
4.4 Sentencias de Control de Flujo 31
4.4.1 break: Salida Inmediata 31
4.4.2 continue: Saltar Iteración 31
4.5 Ejercicio Integrador 32

5 Funciones: Modularidad y Reutilización 35
5.1 Motivación 35
5.2 Anatomía de una Función 35
5.3 Paso de Parámetros 36
5.3.1 Paso por Valor 36
5.3.2 Paso por Referencia 36
5.4 Múltiples Valores de Retorno 37
5.4.1 Usando Parámetros de Salida 37
5.5 Biblioteca Matemática: cmath 38
5.6 Sistema Modular Completo 38

6 Ejercicios Prácticos con Enfoque Estadístico 41
6.1 Ejercicio 1: Normalización Min-Max 41
6.2 Ejercicio 2: Detección de Outliers con Z-Score 42
6.3 Ejercicio 3: Clasificación Multinivel 43
6.4 Ejercicio 4: Generador de Números Aleatorios 44

7 Conclusiones 47

Introducción

Este documento representa el aprendizaje durante el curso de Lenguaje de Programación 1. A lo largo del semestre se ha desarrollado la capacidad de entender la lógica detrás de cada instrucción y cómo estas se traducen en soluciones computacionales aplicadas.

El enfoque del material conecta los conceptos de programación con aplicaciones estadísticas, considerando fundamental el dominio de herramientas computacionales para el análisis de datos en la formación de ingenieros estadísticos e informáticos.

La estructura del documento comienza desde la historia de la computación, continúa con los fundamentos teóricos y finaliza con ejemplos prácticos resueltos. Cada ejercicio incluye comentarios explicativos que facilitan la comprensión de los conceptos presentados.

Capítulo 1

Orígenes de la Computación y Programación

1.1. Los Pioneros de la Computación

1.1.1. Charles Babbage (1791-1871): El Visionario Mecánico

Charles Babbage, matemático británico, diseñó la Máquina Analítica en 1837, un dispositivo mecánico programable que nunca llegó a construirse completamente durante su vida. Esta máquina conceptual contenía todos los elementos fundamentales de una computadora moderna:

- **Almacén**: Equivalente a la memoria RAM actual
- **Molino**: Unidad de procesamiento aritmético (similar a la CPU)
- **Entrada**: Mediante tarjetas perforadas
- **Salida**: Impresión de resultados

El concepto de separar el almacenamiento de datos del procesamiento, planteado hace casi 200 años, constituye un principio arquitectónico vigente en la actualidad.

1.1.2. Ada Lovelace (1815-1852): La Primera Programadora

Augusta Ada King, condesa de Lovelace, escribió en 1843 lo que se considera el primer algoritmo informático de la historia, diseñado específicamente para calcular los números de Bernoulli en la Máquina Analítica.

La contribución revolucionaria de Lovelace fue comprender que las máquinas podían hacer más que cálculos numéricos. Vislumbró que podrían manipular símbolos y procesar cualquier información representable simbólicamente, anticipando conceptos fundamentales de la computación moderna.

1.1.3. Alan Turing (1912-1954): El Padre Teórico

Alan Turing formalizó el concepto de computación mediante su Máquina de Turing (1936), un modelo matemático abstracto que define qué significa que algo sea "computable". Durante la Segunda Guerra Mundial, lideró el equipo que descifró Enigma, contribuyendo significativamente al esfuerzo bélico aliado.

El Test de Turing continúa siendo relevante en el campo de la inteligencia artificial, planteando la cuestión de si una máquina puede exhibir comportamiento inteligente indistinguible del humano.

1.1.4. John von Neumann (1903-1957): La Arquitectura Moderna

Von Neumann propuso en 1945 la arquitectura que lleva su nombre, diseño fundamental de prácticamente todas las computadoras actuales. Su idea revolucionaria fue almacenar tanto el programa como los datos en la misma memoria, permitiendo que las computadoras fueran reprogramables sin modificar el hardware.

Esta arquitectura sigue siendo la base de los sistemas computacionales modernos, demostrando la vigencia de sus fundamentos tras ocho décadas.

1.2. Evolución de los Lenguajes de Programación

1.2.1. De las Máquinas a los Humanos

Los lenguajes de programación han evolucionado para acercarse progresivamente al lenguaje humano, manteniendo la precisión requerida por las máquinas:

Primera Generación - Lenguaje Máquina (años 1940)

Programación directa en binario (0s y 1s). Ejemplo de instrucción:

```
10110000 01100001
```

Extremadamente difícil y propenso a errores. Cada arquitectura tenía su propio código máquina específico.

Segunda Generación - Ensamblador (años 1950)

Uso de mnemónicos más legibles:

```
MOV AX, 5
ADD AX, 3
```

Aunque dependiente del hardware, resultaba más manejable que el código máquina.

Tercera Generación - Lenguajes de Alto Nivel (1950-presente)

Aparición de FORTRAN (1957), COBOL (1959), C (1972), C++ (1985), Python (1991), permitiendo escribir código legible por humanos.

1.3. Lenguajes de Programación más Usados

1.3.1. Índices Estadísticos Internacionales

Para identificar los lenguajes de programación más usados a nivel mundial, se emplean índices estadísticos desarrollados por organizaciones reconocidas del sector tecnológico. Uno de los más importantes es el Índice TIOBE, el cual se actualiza mensualmente y mide la popularidad de los lenguajes de programación considerando la cantidad de profesionales calificados, cursos disponibles y referencias en diversos motores de búsqueda y plataformas web. Este índice permite observar la evolución de los lenguajes y detectar tendencias generales del mercado tecnológico.

Otra fuente estadística relevante es el informe Octoverse de GitHub, el cual analiza el uso real de los lenguajes de programación a partir de millones de repositorios de código abierto alojados en dicha plataforma. A diferencia del Índice TIOBE, el informe Octoverse se centra en la actividad efectiva de los desarrolladores, mostrando cuáles son los lenguajes más utilizados en proyectos reales y colaborativos. En conjunto, ambas fuentes ofrecen una visión complementaria sobre la popularidad y el uso práctico de los lenguajes de programación.

1.3.2. Lenguaje de Programación Líder

De acuerdo con el Índice TIOBE de enero de 2026, como se observa en la Figura 1.1, Python se mantiene como el lenguaje de programación más popular a nivel mundial, seguido por lenguajes tradicionales como C, Java y C++. Asimismo, el informe reconoce a C# como el Lenguaje de Programación del Año 2025 debido a su notable crecimiento interanual, resultado de su constante evolución, su carácter multiplataforma y su adopción como software de código abierto.

Por otro lado, el informe Octoverse 2022 de GitHub posiciona a JavaScript como el lenguaje más utilizado dentro de la plataforma, seguido por Python y Java, lo que evidencia su fuerte presencia en proyectos de desarrollo web y aplicaciones modernas. Esta diferencia entre los resultados de ambas fuentes se explica por el enfoque de cada una: mientras el Índice TIOBE mide la popularidad general, Octoverse refleja el uso real en proyectos de software. Por ello, el liderazgo de un lenguaje depende del criterio estadístico empleado.

A partir del análisis de las estadísticas proporcionadas por el Índice TIOBE y el informe Octoverse de GitHub, se puede concluir que la popularidad los lenguajes de programación varían según el criterio de evaluación empleado. Mientras el Índice TIOBE refleja la popularidad general de los lenguajes a nivel mundial, destacando a Python como el más popular y a C# por su notable crecimiento durante 2025, el informe Octoverse evidencia el uso real en proyectos de software, posicionando a JavaScript como el lenguaje más utilizado en repositorios de código abierto.

En conclusión, no existe un único lenguaje de programación dominante en todos los contextos, sino que cada lenguaje ocupa un lugar relevante según su aplicación y entorno de uso, lo que resalta la importancia de considerar múltiples fuentes estadísticas para comprender las tendencias actuales del desarrollo de software.

Jan 2026	Jan 2025	Change	Programming Language	Ratings	Change
1	1		Python	22.61%	-0.68%
2	4	^	C	10.99%	+2.13%
3	3		Java	8.71%	-1.44%
4	2	v	C++	8.67%	-1.62%
5	5		C#	7.39%	+2.94%
6	6		JavaScript	3.03%	-1.17%
7	9	^	Visual Basic	2.41%	+0.04%
8	8		SQL	2.27%	-0.14%
9	11	^	Delphi/Object Pascal	1.98%	+0.19%
10	18	^^	R	1.82%	+0.81%

Figura 1.1: Ranking de popularidad de lenguajes de programación según el Índice TIOBE (enero de 2026).

Jan 2026	Jan 2025	Change	Programming Language	Ratings	Change
1	1		Python	22.61%	-0.68%
2	4	^	C	10.99%	+2.13%
3	3		Java	8.71%	-1.44%
4	2	v	C++	8.67%	-1.62%
5	5		C#	7.39%	+2.94%
6	6		JavaScript	3.03%	-1.17%
7	9	^	Visual Basic	2.41%	+0.04%
8	8		SQL	2.27%	-0.14%
9	11	^	Delphi/Object Pascal	1.98%	+0.19%
10	18	^^	R	1.82%	+0.81%

Figura 1.2: Enter Caption

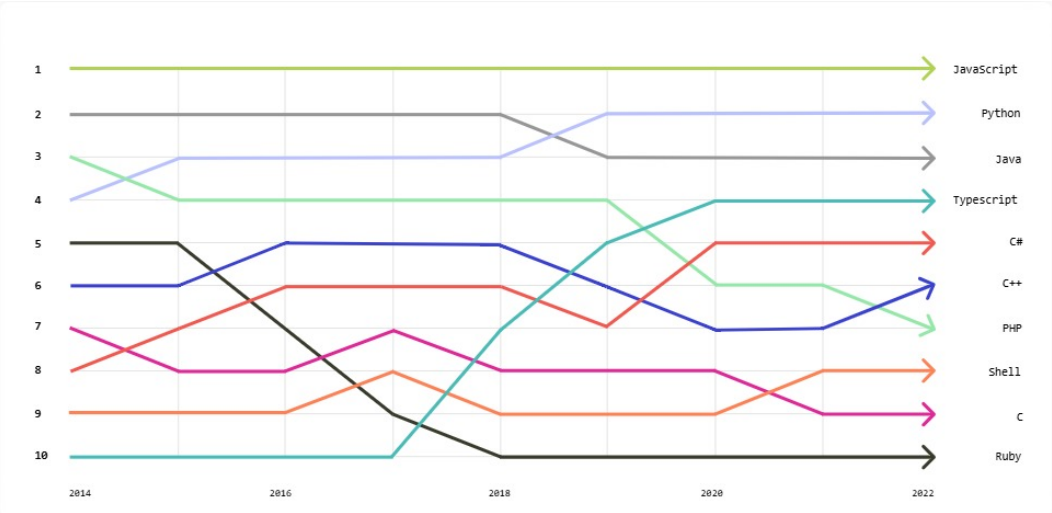

Figura 1.3: Lenguajes de programación más utilizados en GitHub según el informe Octoverse 2022.

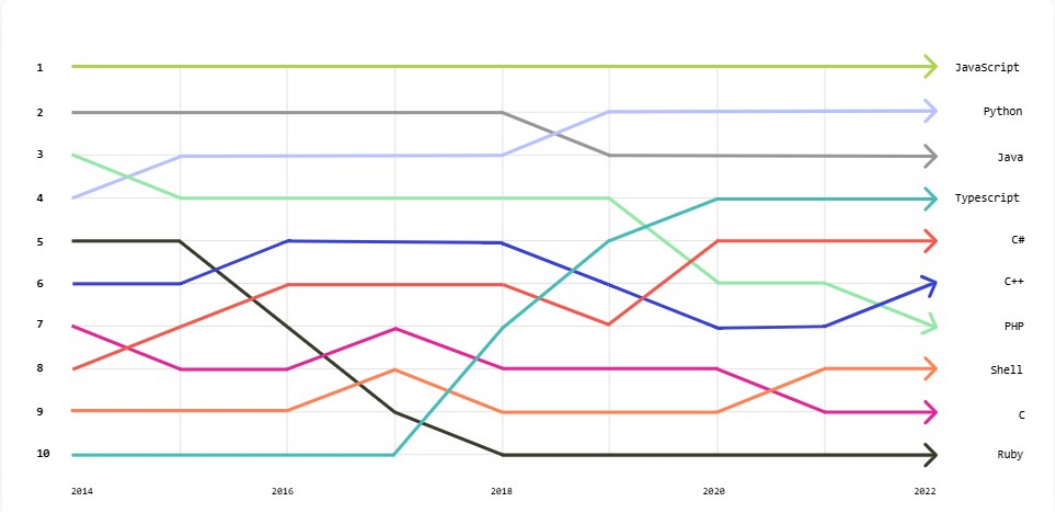

Figura 1.4: Lenguajes de programación más utilizados en GitHub según el informe Octoverse 2022

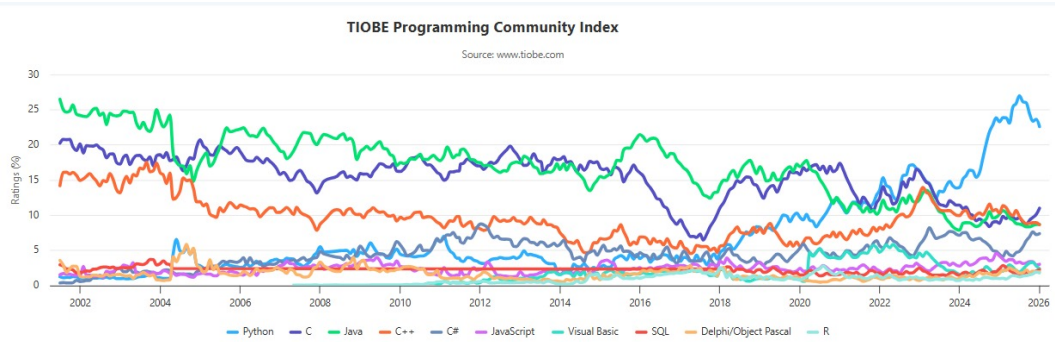

Figura 1.5: TIOBE Programming Community index

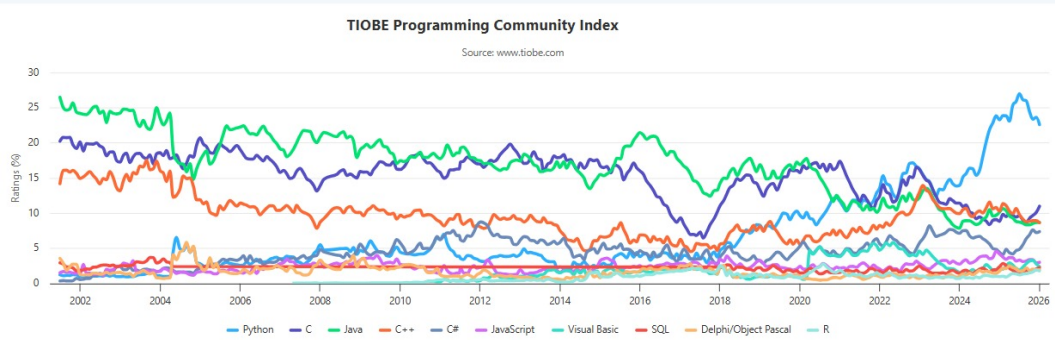

Figura 1.6: TIOBE Programming Community index

1.4. Paradigmas de Programación

Un paradigma de programación es una filosofía o estilo para estructurar y escribir código.

1.4.1. Paradigma Imperativo

Consiste en especificar a la computadora cómo realizar una tarea, paso por paso.

```
// Sumar elementos de un array
int suma = 0;
for (int i = 0; i < n; i++) {
    suma = suma + datos[i];
}
```

Listing 1.1: Suma de elementos - Paradigma Imperativo

1.4.2. Paradigma Declarativo

Se especifica qué se desea obtener, no cómo obtenerlo.

```
SELECT nombre, promedio
FROM estudiantes
WHERE promedio > 14;
```

Listing 1.2: Consulta declarativa en SQL

1.4.3. Paradigma Orientado a Objetos (POO)

Organiza el código en "objetos" que encapsulan atributos (características) y métodos (comportamientos).

```
class Estudiante {
private:
    string nombre;
    double promedio;
public:
    void calcularPromedio(vector<double> notas) {
        double suma = 0;
        for(double nota : notas) suma += nota;
        promedio = suma / notas.size();
    }

    string getCategoria() {
        if(promedio >= 14) return "Aprobado";
        else return "Desaprobado";
    }
};
```

Listing 1.3: Ejemplo de Clase en POO

1.5. Lógica Proposicional

1.5.1. Operadores Lógicos Fundamentales

A	B	A AND B	A OR B	NOT A
V	V	V	V	F
V	F	F	V	F
F	V	F	V	V
F	F	F	F	V

Cuadro 1.1: Tabla de verdad completa

Aplicación práctica en estadística:

```
// Filtrar datos at picos
if ((valor > Q3 + 1.5*IQR) || (valor < Q1 - 1.5*IQR)) {
    cout << "Outlier detectado: " << valor << endl;
}
```

Listing 1.4: Detección de outliers usando operadores lógicos

Capítulo 2

Introducción a C y C++

2.1. Relevancia de C y C++ en 2026

C y C++ mantienen su relevancia por las siguientes razones:

1. **Fundamento de sistemas**: Linux, Windows y MacOS están escritos en C. Python está implementado en C.
2. **Rendimiento**: Para procesamiento de grandes volúmenes de datos, C/C++ ofrecen rendimiento superior.
3. **Control preciso**: Permiten gestionar memoria, optimizar recursos y acceder directamente al hardware.
4. **Formación sólida**: Desarrollan disciplina en el manejo de tipos de datos, gestión de memoria y eficiencia.

2.2. C: El Lenguaje del Sistema

2.2.1. Historia de C

Desarrollado por Dennis Ritchie entre 1969-1973 en los Bell Labs para reescribir UNIX. C proporcionó:

- Abstracción suficiente para ser portable
- Eficiencia comparable al ensamblador
- Estructuras de control modernas (if, while, for)
- Gestión directa de memoria mediante punteros

2.2.2. Características de C

- **Tipado estático**: Declaración explícita del tipo de cada variable
- **Compilado**: Traducción a código máquina antes de la ejecución
- **Procedimental**: Organizado en funciones
- **Acceso a bajo nivel**: Punteros y aritmética de direcciones

2.3. C++: Extensión de C

2.3.1. Historia de C++

Creado por Bjarne Stroustrup en 1979 (inicialmente "C con Clases"). Mantiene la eficiencia de C añadiendo abstracciones modernas:

- Clases y objetos (POO)
- Templates (programación genérica)
- Manejo de excepciones
- Referencias
- Sobrecarga de operadores
- Standard Template Library (STL)

2.3.2. Diferencias Clave entre C y C++

C	C++
Solo paradigma procedimental	Multiparadigma (procedimental, POO, genérico)
printf/scanf para E/S	iostream (cin/cout) más seguro
malloc/free para memoria dinámica	new/delete + constructores/destructores
struct sin métodos	class con métodos y encapsulación
Sin sobrecarga de funciones	Sobrecarga de funciones y operadores

Cuadro 2.1: Comparación C vs C++

2.4. Primer Programa en C

```c
#include <stdio.h>

int main() {
    printf("Hola desde C!\n");
    printf("Soy Dina Delgado, estudiante de FINESI\n");
    return 0;
}
```

Listing 2.1: Programa básico en C

Análisis del código:

1. `#include <stdio.h>`: Incluye la biblioteca estándar de entrada/salida.

2. `int main()`: Función principal donde comienza la ejecución.
3. `printf("...")`: Imprime texto en pantalla. `\n` representa salto de línea.
4. `return 0`: Retorna 0 para indicar ejecución exitosa.

2.5. Primer Programa en C++

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hola desde C++!" << endl;
    cout << "Aprendiendo programacion en UNAP" << endl;
    return 0;
}
```

Listing 2.2: Programa básico en C++

Diferencias respecto a C:

- Uso de `iostream` en lugar de `stdio.h`
- `cout` (character output) en lugar de `printf`
- « `es el operador de inserción`
- » `endl equivale a \n con vaciado de buffer`
- » `using namespace std evita prefijo std::`

2.6. Tipos de Datos Fundamentales

Tipo	Bytes	Rango	Uso
char	1	-128 a 127	Caracteres individuales
int	4	$-2{,}1 \times 10^9$ a $2{,}1 \times 10^9$	Enteros
float	4	$\pm 3{,}4 \times 10^{38}$	Decimales (7 dígitos)
double	8	$\pm 1{,}7 \times 10^{308}$	Decimales (15 dígitos)
bool	1	true/false	Valores lógicos (C++)

Cuadro 2.2: Tipos de datos fundamentales

`Criterios de selección:`

- `Contadores, índices` → `int`
- `Dinero, medidas precisas` → `double`
- `Simulaciones físicas` → `double`
- `Optimización extrema de memoria` → `float`

2.6.1. Ejemplo con Diferentes Tipos

```
#include <iostream>
using namespace std;

int main() {
    int codigo = 247092;
    char seccion = 'A';
    double promedio = 15.75;
    bool aprobado = (promedio >= 10.5);

    cout << "=== SISTEMA ACADEMICO  ===" << endl;
    cout << "Codigo: " << codigo << endl;
    cout << "Seccion: " << seccion << endl;
    cout << "Promedio: " << promedio << endl;
    cout << "Estado: " << (aprobado ? "APROBADO" : "
       DESAPROBADO") << endl;

    cout << "\nTamanos en memoria:" << endl;
    cout << "int: " << sizeof(int) << " bytes" << endl;
    cout << "double: " << sizeof(double) << " bytes" <<
       endl;
    cout << "char: " << sizeof(char) << " byte" << endl;

    return 0;
}
```

Listing 2.3: Uso de tipos de datos

Capítulo 3

Operadores

3.1. Operadores Aritméticos

Operador	Operación	Ejemplo
+	Suma	5 + 3 = 8
-	Resta	5 - 3 = 2
*	Multiplicación	5 * 3 = 15
/	División	5 / 3 = 1 (entero)
%	Módulo (residuo)	5 % 3 = 2

Cuadro 3.1: Operadores aritméticos

3.1.1. División Entera

Consideración importante: cuando se dividen dos enteros, C realiza división entera (trunca los decimales). Para obtener resultados decimales, al menos uno de los operandos debe ser float o double.

```c
#include <stdio.h>

int main() {
    int a = 5, b = 2;
    int resultado1 = a / b;            // Division entera
    double resultado2 = a / b;          // Sigue siendo entera
    double resultado3 = (double)a / b; // Conversion a decimal

    printf("5 / 2 con int: %d\n", resultado1);      // 2
    printf("5 / 2 sin cast: %f\n", resultado2);     // 2.000000
    printf("5 / 2 con cast: %f\n", resultado3);     // 2.500000

    return 0;
```

```
}
```

Listing 3.1: División entera vs decimal

3.1.2. El Operador Módulo

El operador módulo (%) devuelve el residuo de una división.

```
#include <iostream>
using namespace std;

int main() {
    int numero;
    cout << "Ingrese un numero: ";
    cin >> numero;

    // Determinar si es par o impar
    if (numero % 2 == 0) {
        cout << numero << " es PAR" << endl;
    } else {
        cout << numero << " es IMPAR" << endl;
    }

    // Obtener el ultimo digito
    int ultimo_digito = numero % 10;
    cout << "Ultimo digito: " << ultimo_digito << endl;

    // Indices circulares
    int indice = 15;
    int indice_circular = indice % 7;
    cout << "Dia de la semana: " << indice_circular <<
       endl;

    return 0;
}
```

Listing 3.2: Aplicaciones del operador módulo

3.2. Operadores Relacionales y Lógicos

3.2.1. Operadores Relacionales

Advertencia: Distinguir entre = (asignación) y == (comparación).

```
int x = 5;
if (x = 3) {  // ERROR: asignacion, no comparacion
    printf("Esto siempre se ejecuta\n");
}

```

Operador	Significado	Ejemplo (a=5, b=3)
==	Igual a	a == b (false)
!=	Diferente de	a != b (true)
¿	Mayor que	a ¿b (true)
¡	Menor que	a ¡b (false)
¿=	Mayor o igual	a ¿= b (true)
¡=	Menor o igual	a ¡= b (false)

Cuadro 3.2: Operadores relacionales

```
if (x == 3) { // CORRECTO: comparacion
    printf("Esto solo si x vale 3\n");
}
```

Listing 3.3: Error común: asignación vs comparación

3.2.2. Operadores Lógicos

Operador	Significado	Ejemplo
&&	AND (y)	(a ¿0) && (b ¿0)
——	OR (o)	(a ¿0) —— (b ¿0)
!	NOT (no)	!(a ¿0)

Cuadro 3.3: Operadores lógicos

3.2.3. Aplicación Estadística

```
#include <iostream>
#include <cmath>
using namespace std;

int main() {
    int edad;
    double ingreso;
    char tiene_titulo;

    cout << "=== ENCUESTA SOCIOECONOMICA ===" << endl;
    cout << "Edad: ";
    cin >> edad;
    cout << "Ingreso mensual (S/.): ";
    cin >> ingreso;
    cout << "Tiene titulo universitario? (S/N): ";
    cin >> tiene_titulo;

    bool es_poblacion_objetivo = (edad >= 25 && edad <=
        60) &&
                                    (ingreso >= 1000 &&
                                       ingreso <= 5000) &&
```

```
                                        (tiene_titulo == 'S' ||
                                            tiene_titulo == 's'
                                        );

    if (es_poblacion_objetivo) {
        cout << "\nPerfil seleccionado para el estudio!" << endl;
    } else {
        cout << "\nNo cumple criterios de inclusion." << endl;

        if (edad < 25 || edad > 60) {
            cout << "- Edad fuera del rango (25-60)" << endl;
        }
        if (ingreso < 1000 || ingreso > 5000) {
            cout << "- Ingreso fuera del rango (1000-5000)" << endl;
        }
        if (tiene_titulo != 'S' && tiene_titulo != 's') {
            cout << "- Requiere titulo universitario" << endl;
        }
    }

    return 0;
}
```

Listing 3.4: Filtros con operadores lógicos

3.3. Operadores de Incremento y Decremento

3.3.1. Prefijo vs Postfijo

```
#include <iostream>
using namespace std;

int main() {
    int contador = 5;

    cout << "=== POST-INCREMENTO (contador++) ===" << endl;
    cout << "Valor de contador++: " << contador++ << endl; // Muestra 5
    cout << "Ahora contador vale: " << contador << endl; // Muestra 6

    cout << "\n=== PRE-INCREMENTO (++contador) ===" << endl;
```

```
12      contador = 5;
13      cout << "Valor de ++contador: " << ++contador << endl
           ; // Muestra 6
14      cout << "Ahora contador vale: " << contador << endl;
           // Muestra 6
15
16      return 0;
17  }
```

Listing 3.5: Diferencia entre prefijo y postfijo

Regla:

- contador++: Usa primero, incrementa después (POSTfijo)
- ++contador: Incrementa primero, usa después (PREfijo)

3.3.2. Uso en Bucles

```
1  // En bucles, ambas formas producen el mismo resultado
2  for (int i = 0; i < 10; i++) {
3      printf("%d ", i);
4  }
5
6  for (int i = 0; i < 10; ++i) {
7      printf("%d ", i);
8  }
```

Listing 3.6: Incremento en bucles

3.4. Operadores de Asignación Compuesta

Operador	Equivalente	Ejemplo
+=	a = a + b	suma += 5
-=	a = a - b	total -= impuesto
*=	a = a * b	precio *= 1.18
/=	a = a / b	promedio /= n
%=	a = a % b	indice %= tamaño

Cuadro 3.4: Operadores de asignación compuesta

3.4.1. Cálculo de Estadísticas

```
#include <iostream>
#include <vector>
#include <cmath>
using namespace std;

int main() {
    vector<double> datos = {23.5, 25.1, 24.8, 26.2,
       23.9};

    double suma = 0;
    for (double valor : datos) {
        suma += valor;
    }
    double media = suma / datos.size();

    double suma_cuadrados = 0;
    for (double valor : datos) {
        double diferencia = valor - media;
        suma_cuadrados += diferencia * diferencia;
    }
    double varianza = suma_cuadrados / datos.size();

    cout << "Media: " << media << endl;
    cout << "Varianza: " << varianza << endl;
    cout << "Desviacion estandar: " << sqrt(varianza) <<
       endl;

    return 0;
}
```

Listing 3.7: Estadísticas con operadores compuestos

Capítulo 4

Estructuras de Control

4.1. Introducción

Las estructuras de control permiten que los programas tomen decisiones, repitan tareas y se adapten a diferentes situaciones. Constituyen las herramientas fundamentales para expresar la lógica computacional.

4.2. Estructuras Selectivas

4.2.1. if: La Decisión Simple

```
#include <iostream>
using namespace std;

int main() {
    double temperatura;

    cout << "Ingrese temperatura corporal (C): ";
    cin >> temperatura;

    if (temperatura > 37.5) {
        cout << "ALERTA: Posible fiebre detectada" << endl;
        cout << "Consulte a un medico" << endl;
    }

    cout << "Fin del chequeo" << endl;
    return 0;
}
```

Listing 4.1: Estructura if simple

El bloque if se ejecuta únicamente cuando la condición es verdadera.

4.2.2. if-else: Dos Caminos

```
#include <iostream>
using namespace std;

int main() {
    double nota;

    cout << "Ingrese su nota final: ";
    cin >> nota;

    if (nota >= 10.5) {
        cout << "APROBADO!" << endl;
        cout << "Felicitaciones por tu esfuerzo" << endl;
    } else {
        cout << "DESAPROBADO" << endl;
        cout << "Se recomienda reforzar el estudio" <<
           endl;
    }

    return 0;
}
```

Listing 4.2: Estructura if-else

Siempre se ejecuta exactamente uno de los dos bloques.

4.2.3. if-else if-else: Decisiones Múltiples

```
#include <iostream>
using namespace std;

int main() {
    double imc;

    cout << "Ingrese su IMC: ";
    cin >> imc;

    cout << "\n=== CLASIFICACION OMS ===" << endl;

    if (imc < 18.5) {
        cout << "Bajo peso" << endl;
        cout << "Recomendacion: Consulte nutricionista"
           << endl;
    }
    else if (imc >= 18.5 && imc < 25.0) {
        cout << "Peso normal" << endl;
        cout << "Mantengase saludable" << endl;
    }
    else if (imc >= 25.0 && imc < 30.0) {
        cout << "Sobrepeso" << endl;
```

```
22          cout << "Considere mejorar habitos alimenticios"
               << endl;
23      }
24      else if (imc >= 30.0 && imc < 35.0) {
25          cout << "Obesidad Grado I" << endl;
26          cout << "Consulte a su medico" << endl;
27      }
28      else {
29          cout << "Obesidad Grado II o superior" << endl;
30          cout << "Atencion medica prioritaria" << endl;
31      }
32
33      return 0;
34  }
```

Listing 4.3: Clasificación del IMC

Las condiciones se evalúan secuencialmente. Al encontrar una condición verdadera, se ejecuta ese bloque y se omiten los restantes.

4.2.4. switch: Opciones Discretas

```
1   #include <iostream>
2   using namespace std;
3
4   int main() {
5       int opcion;
6
7       cout << "=== CALCULADORA ESTADISTICA ===" << endl;
8       cout << "1. Media" << endl;
9       cout << "2. Mediana" << endl;
10      cout << "3. Moda" << endl;
11      cout << "4. Desviacion estandar" << endl;
12      cout << "5. Varianza" << endl;
13      cout << "0. Salir" << endl;
14      cout << "\nSeleccione opcion: ";
15      cin >> opcion;
16
17      switch (opcion) {
18          case 1:
19              cout << "Calculando media..." << endl;
20              break;
21          case 2:
22              cout << "Calculando mediana..." << endl;
23              break;
24          case 3:
25              cout << "Calculando moda..." << endl;
26              break;
27          case 4:
28              cout << "Calculando desviacion estandar..."
                   << endl;
```

```
            break;
        case 5:
            cout << "Calculando varianza..." << endl;
            break;
        case 0:
            cout << "Saliendo del programa..." << endl;
            break;
        default:
            cout << "Opcion invalida." << endl;
    }

    return 0;
}
```

Listing 4.4: Menú con switch

La instrucción break es esencial para evitar la ejecución de casos subsiguientes (fall-through).

4.3. Estructuras Repetitivas (Bucles)

4.3.1. El Bucle while

```
#include <iostream>
using namespace std;

int main() {
    int numero, intentos = 0;
    const int SECRET = 42;

    cout << "=== JUEGO: ADIVINA EL NUMERO ===" << endl;

    while (numero != SECRET && intentos < 5) {
        cout << "Intento " << (intentos + 1) << ": ";
        cin >> numero;

        if (numero < SECRET) {
            cout << "Muy bajo" << endl;
        } else if (numero > SECRET) {
            cout << "Muy alto" << endl;
        }

        intentos++;
    }

    if (numero == SECRET) {
        cout << "CORRECTO! " << intentos + 1 << "
           intentos" << endl;
    } else {
```

```
26          cout << "Intentos agotados. Era " << SECRET <<
               endl;
27      }
28
29      return 0;
30  }
```

Listing 4.5: Juego con while

La condición se verifica antes de cada iteración. Si es falsa inicialmente, el bucle no se ejecuta.

4.3.2. El Bucle for

```
1  #include <iostream>
2  #include <iomanip>
3  using namespace std;
4
5  int main() {
6      int n;
7
8      cout << "Tamano de tabla de multiplicar: ";
9      cin >> n;
10
11     cout << "\n=== TABLA DE MULTIPLICAR ===" << endl;
12
13     for (int i = 1; i <= n; i++) {
14         for (int j = 1; j <= n; j++) {
15             cout << setw(4) << (i * j);
16         }
17         cout << endl;
18     }
19
20     return 0;
21 }
```

Listing 4.6: Tabla de multiplicar con for

Estructura del for:

1. Inicialización: Se ejecuta una vez al inicio
2. Condición: Se verifica antes de cada iteración
3. Actualización: Se ejecuta después de cada iteración

4.3.3. El Bucle do-while

```
#include <iostream>
using namespace std;

int main() {
    double dato;
    double suma = 0;
    int contador = 0;
    char continuar;

    cout << "=== CALCULADORA DE MEDIA ===" << endl;

    do {
        cout << "Ingrese un dato: ";
        cin >> dato;

        suma += dato;
        contador++;

        cout << "Desea ingresar otro dato? (s/n): ";
        cin >> continuar;

    } while (continuar == 's' || continuar == 'S');

    double media = suma / contador;
    cout << "\nMedia de " << contador << " datos: " <<
       media << endl;

    return 0;
}
```

Listing 4.7: Calculadora con do-while

Característica: Se ejecuta al menos una vez, ya que la condición se verifica al final.

4.3.4. Comparación de Bucles

Bucle	Cuándo usar	Ejemplo típico
while	Número de iteraciones desconocido	Leer hasta EOF
for	Número de iteraciones conocido	Recorrer array
do-while	Ejecutar al menos una vez	Menús, validación

Cuadro 4.1: Comparación de estructuras repetitivas

4.4. Sentencias de Control de Flujo

4.4.1. break: Salida Inmediata

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<double> datos = {23.5, 25.1, 99.9, 24.8,
       26.2};
    double suma = 0;
    int count = 0;

    cout << "Procesando datos..." << endl;

    for (double valor : datos) {
        if (valor > 50) {
            cout << "DATO ANOMALO: " << valor << endl;
            cout << "Deteniendo procesamiento" << endl;
            break;
        }

        suma += valor;
        count++;
    }

    if (count > 0) {
        cout << "Datos validos: " << count << endl;
        cout << "Media parcial: " << suma / count << endl
           ;
    }

    return 0;
}
```

Listing 4.8: Uso de break

4.4.2. continue: Saltar Iteración

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<double> datos = {23.5, -999, 25.1, -999, 24.8,
        26.2, -999};
    double suma = 0;
    int datos_validos = 0;
```

```
    cout << "Procesando dataset con valores faltantes..." << endl;

    for (int i = 0; i < datos.size(); i++) {
        if (datos[i] == -999) {
            cout << "Posicion " << i << ": faltante (omitido)" << endl;
            continue;
        }

        suma += datos[i];
        datos_validos++;
        cout << "Posicion " << i << ": " << datos[i] << endl;
    }

    cout << "\nDatos validos: " << datos_validos << " de " << datos.size() << endl;
    cout << "Media: " << suma / datos_validos << endl;

    return 0;
}
```

Listing 4.9: Uso de continue

4.5. Ejercicio Integrador

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<double> notas;
    int num_estudiantes;

    cout << "Cuantos estudiantes hay en el curso? ";
    cin >> num_estudiantes;

    for (int i = 0; i < num_estudiantes; i++) {
        double nota;

        do {
            cout << "Nota del estudiante " << (i+1) << " (0-20): ";
            cin >> nota;

            if (nota < 0 || nota > 20) {
```

```
                    cout << "Nota invalida. Rango: 0-20" << endl;
                }
            } while (nota < 0 || nota > 20);

            notas.push_back(nota);
        }

        double suma = 0;
        double nota_max = notas[0];
        double nota_min = notas[0];
        int aprobados = 0;

        for (double nota : notas) {
            suma += nota;

            if (nota > nota_max) nota_max = nota;
            if (nota < nota_min) nota_min = nota;

            if (nota >= 10.5) aprobados++;
        }

        double promedio = suma / notas.size();
        double porcentaje = (aprobados * 100.0) / notas.size();

        cout << "\n=== REPORTE ESTADISTICO ===" << endl;
        cout << "Estudiantes: " << notas.size() << endl;
        cout << "Promedio: " << promedio << endl;
        cout << "Nota maxima: " << nota_max << endl;
        cout << "Nota minima: " << nota_min << endl;
        cout << "Aprobados: " << aprobados << " (" << porcentaje << "%)" << endl;
        cout << "Desaprobados: " << (notas.size() - aprobados) << endl;

        if (porcentaje >= 80) {
            cout << "\nRendimiento: EXCELENTE" << endl;
        } else if (porcentaje >= 60) {
            cout << "\nRendimiento: BUENO" << endl;
        } else if (porcentaje >= 40) {
            cout << "\nRendimiento: REGULAR" << endl;
        } else {
            cout << "\nRendimiento: DEFICIENTE" << endl;
        }

        return 0;
    }
```

Listing 4.10: Sistema de análisis de notas

Capítulo 5

Funciones: Modularidad y Reutilización

5.1. Motivación

Las funciones permiten escribir código una sola vez y reutilizarlo múltiples veces, evitando repetición.

Ventajas:

- Evitar repetición: Principio DRY (Don't Repeat Yourself)
- Organización: Cada función tiene un propósito específico
- Mantenimiento: Correcciones en un solo lugar
- Abstracción: Uso sin conocer detalles internos
- Testing: Pruebas independientes

5.2. Anatomía de una Función

```
// Declaracion (prototipo)
double calcular_media(vector<double> datos);

int main() {
    vector<double> temperaturas = {23.5, 25.1, 24.8};
    double media = calcular_media(temperaturas);
    cout << "Media: " << media << endl;
    return 0;
}

// Definicion
double calcular_media(vector<double> datos) {
    double suma = 0;
    for (double valor : datos) {
```

```
15          suma += valor;
16      }
17      return suma / datos.size();
18  }
```

Listing 5.1: Estructura de una función

```
Componentes:

  1. Tipo de retorno: Tipo de dato que devuelve

  2. Nombre: Identificador descriptivo

  3. Parámetros: Datos de entrada

  4. Cuerpo: Implementación

  5. return: Valor de retorno
```

5.3. Paso de Parámetros

5.3.1. Paso por Valor

```
1  #include <iostream>
2  using namespace std;
3
4  void intentar_modificar(int x) {
5      x = 100;
6      cout << "Dentro de funcion: " << x << endl;
7  }
8
9  int main() {
10     int numero = 5;
11     cout << "Antes: " << numero << endl;
12
13     intentar_modificar(numero);
14
15     cout << "Despues: " << numero << endl; // Sigue
          siendo 5
16     return 0;
17 }
```

Listing 5.2: Paso por valor (copia)

5.3.2. Paso por Referencia

```
#include <iostream>
using namespace std;

void modificar_real(int& x) {
    x = 100;
    cout << "Dentro de funcion: " << x << endl;
}

int main() {
    int numero = 5;
    cout << "Antes: " << numero << endl;

    modificar_real(numero);

    cout << "Despues: " << numero << endl; // Ahora es
        100
    return 0;
}
```

Listing 5.3: Paso por referencia

Criterio de selección:

- Paso por valor: No modificar el original
- Paso por referencia: Modificar el original o evitar copia costosa

5.4. Múltiples Valores de Retorno

5.4.1. Usando Parámetros de Salida

```
#include <iostream>
#include <vector>
using namespace std;

void calcular_estadisticas(vector<double> datos,
                           double& media,
                           double& minimo,
                           double& maximo) {
    if (datos.empty()) return;

    double suma = 0;
    minimo = maximo = datos[0];

    for (double valor : datos) {
        suma += valor;
        if (valor < minimo) minimo = valor;
        if (valor > maximo) maximo = valor;
    }
```

```
19
20      media = suma / datos.size();
21  }
22
23  int main() {
24      vector<double> temps = {23.5, 25.1, 24.8, 26.2,
           23.9};
25      double media, min, max;
26
27      calcular_estadisticas(temps, media, min, max);
28
29      cout << "Media: " << media << endl;
30      cout << "Minimo: " << min << endl;
31      cout << "Maximo: " << max << endl;
32
33      return 0;
34  }
```

Listing 5.4: Función con múltiples salidas

5.5. Biblioteca Matemática: cmath

```
1  #include <iostream>
2  #include <cmath>
3  using namespace std;
4
5  int main() {
6      double x = 16.0;
7
8      cout << "=== FUNCIONES MATEMATICAS ===" << endl;
9      cout << "sqrt(16) = " << sqrt(x) << endl;
10     cout << "pow(2, 3) = " << pow(2, 3) << endl;
11     cout << "exp(1) = " << exp(1) << endl;
12     cout << "log(10) = " << log(10) << endl;
13     cout << "log10(100) = " << log10(100) << endl;
14     cout << "abs(-5) = " << abs(-5) << endl;
15     cout << "ceil(3.2) = " << ceil(3.2) << endl;
16     cout << "floor(3.8) = " << floor(3.8) << endl;
17     cout << "round(3.5) = " << round(3.5) << endl;
18
19     return 0;
20 }
```

Listing 5.5: Funciones matemáticas

5.6. Sistema Modular Completo

```
#include <iostream>
#include <vector>
#include <cmath>
#include <algorithm>
using namespace std;

double calcular_media(const vector<double>& datos);
double calcular_varianza(const vector<double>& datos);
double calcular_desviacion(const vector<double>& datos);
double calcular_mediana(vector<double> datos);
void mostrar_resumen(const vector<double>& datos);

int main() {
    vector<double> dataset = {23.5, 25.1, 24.8, 26.2,
       23.9, 27.1, 24.5};

    cout << "=== ANALISIS ESTADISTICO ===" << endl;
    cout << "Dataset: ";
    for (double x : dataset) cout << x << " ";
    cout << "\n" << endl;

    mostrar_resumen(dataset);

    return 0;
}

double calcular_media(const vector<double>& datos) {
    double suma = 0;
    for (double x : datos) suma += x;
    return suma / datos.size();
}

double calcular_varianza(const vector<double>& datos) {
    double media = calcular_media(datos);
    double suma_cuadrados = 0;

    for (double x : datos) {
        double diferencia = x - media;
        suma_cuadrados += diferencia * diferencia;
    }

    return suma_cuadrados / datos.size();
}

double calcular_desviacion(const vector<double>& datos) {
    return sqrt(calcular_varianza(datos));
}

double calcular_mediana(vector<double> datos) {
    sort(datos.begin(), datos.end());
    int n = datos.size();
```

```
51
52      if (n % 2 == 0) {
53          return (datos[n/2 - 1] + datos[n/2]) / 2.0;
54      } else {
55          return datos[n/2];
56      }
57  }
58
59  void mostrar_resumen(const vector<double>& datos) {
60      cout << "N = " << datos.size() << endl;
61      cout << "Media = " << calcular_media(datos) << endl;
62      cout << "Mediana = " << calcular_mediana(datos) <<
           endl;
63      cout << "Varianza = " << calcular_varianza(datos) <<
           endl;
64      cout << "Desv. Std = " << calcular_desviacion(datos)
           << endl;
65
66      double min = *min_element(datos.begin(), datos.end())
           ;
67      double max = *max_element(datos.begin(), datos.end())
           ;
68
69      cout << "Minimo = " << min << endl;
70      cout << "Maximo = " << max << endl;
71      cout << "Rango = " << (max - min) << endl;
72  }
```

Listing 5.6: Análisis estadístico modular

Capítulo 6

Ejercicios Prácticos con Enfoque Estadístico

6.1. Ejercicio 1: Normalización Min-Max

Normalizar un conjunto de datos al rango [0,1].

Fórmula: $x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}}$

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<double> edades = {25, 30, 45, 22, 35, 28};
    vector<double> edades_norm;

    double edad_min = *min_element(edades.begin(), edades.end());
    double edad_max = *max_element(edades.begin(), edades.end());

    cout << "Datos originales: ";
    for (double x : edades) cout << x << " ";
    cout << endl;

    for (double edad : edades) {
        double normalizado = (edad - edad_min) / (edad_max - edad_min);
        edades_norm.push_back(normalizado);
    }

    cout << "Datos normalizados: ";
    for (double x : edades_norm) {
        cout << fixed << x << " ";
    }
    cout << endl;
```

```

    return 0;
}
```

Listing 6.1: Normalización Min-Max

6.2. Ejercicio 2: Detección de Outliers con Z-Score

Criterio: Un dato es outlier si $|z| > 3$ donde $z = \frac{x-\mu}{\sigma}$

```
#include <iostream>
#include <vector>
#include <cmath>
using namespace std;

double calcular_media(const vector<double>& datos) {
    double suma = 0;
    for (double x : datos) suma += x;
    return suma / datos.size();
}

double calcular_desviacion(const vector<double>& datos) {
    double media = calcular_media(datos);
    double suma_cuad = 0;

    for (double x : datos) {
        suma_cuad += pow(x - media, 2);
    }

    return sqrt(suma_cuad / datos.size());
}

int main() {
    vector<double> datos = {23.5, 25.1, 24.8, 95.0, 26.2,
        23.9};

    double media = calcular_media(datos);
    double desv = calcular_desviacion(datos);

    cout << "Media: " << media << endl;
    cout << "Desv. Std: " << desv << endl;
    cout << "\nAnalisis de outliers:" << endl;

    for (int i = 0; i < datos.size(); i++) {
        double z_score = (datos[i] - media) / desv;

        cout << "Dato " << i+1 << ": " << datos[i];
        cout << " (z = " << z_score << ")";
```

```

        if (abs(z_score) > 3) {
            cout << " <-- OUTLIER";
        }
        cout << endl;
    }

    return 0;
}
```

Listing 6.2: Detección de outliers

6.3. Ejercicio 3: Clasificación Multinivel

```
#include <iostream>
using namespace std;

int main() {
    double indice;

    cout << "Indice de calidad del aire (0-500): ";
    cin >> indice;

    cout << "\n=== CLASIFICACION EPA ===" << endl;

    if (indice <= 50) {
        cout << "Categoria: BUENA" << endl;
        cout << "Color: Verde" << endl;
        cout << "Accion: Ninguna restriccion" << endl;
    }
    else if (indice <= 100) {
        cout << "Categoria: MODERADA" << endl;
        cout << "Color: Amarillo" << endl;
        cout << "Accion: Sensibles reducir ejercicio" <<
            endl;
    }
    else if (indice <= 150) {
        cout << "Categoria: INSALUBRE PARA SENSIBLES" <<
            endl;
        cout << "Color: Naranja" << endl;
        cout << "Accion: Limitar actividad exterior" <<
            endl;
    }
    else if (indice <= 200) {
        cout << "Categoria: INSALUBRE" << endl;
        cout << "Color: Rojo" << endl;
        cout << "Accion: Evitar ejercicio intenso" <<
            endl;
    }
```

```
    else if (indice <= 300) {
        cout << "Categoria: MUY INSALUBRE" << endl;
        cout << "Color: Purpura" << endl;
        cout << "Accion: Permanecer en interiores" <<
           endl;
    }
    else {
        cout << "Categoria: PELIGROSO" << endl;
        cout << "Color: Marron" << endl;
        cout << "Accion: EMERGENCIA" << endl;
    }

    return 0;
}
```

Listing 6.3: Clasificación de calidad del aire

6.4. Ejercicio 4: Generador de Números Aleatorios

```
#include <iostream>
#include <random>
#include <vector>
using namespace std;

int main() {
    random_device rd;
    mt19937 gen(rd());
    uniform_real_distribution<> dist(0.0, 1.0);

    int n_muestras = 10000;
    vector<double> muestras;

    for (int i = 0; i < n_muestras; i++) {
        muestras.push_back(dist(gen));
    }

    double suma = 0;
    for (double x : muestras) suma += x;
    double media_empirica = suma / n_muestras;

    cout << "=== SIMULACION MONTE CARLO ===" << endl;
    cout << "Distribucion: Uniforme(0, 1)" << endl;
    cout << "Muestras: " << n_muestras << endl;
    cout << "Media teorica: 0.5" << endl;
    cout << "Media empirica: " << media_empirica << endl;
    cout << "Error: " << abs(media_empirica - 0.5) <<
       endl;

```

```
29        return 0;
30  }
```

Listing 6.4: Simulación Monte Carlo

Capítulo 7

Conclusiones

Este documento presenta los fundamentos de programación en C y C++. Cubre orígenes históricos de la computación, evolución de lenguajes, sintaxis básica, operadores, estructuras de control, funciones y ejercicios prácticos.

La programación en C y C++ es fundamental para informática: desarrolla pensamiento algorítmico, soluciones eficientes y bases para estructuras de datos, algoritmos y programación orientada a objetos.

Dina Milagros Delgado Quispe

Código: 247092

Escuela Profesional de Ingeniería Estadística e Informática

Universidad Nacional del Altiplano - Puno

2026